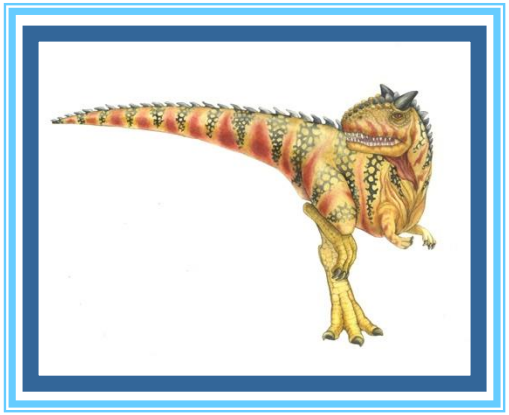
Bölüm 3: Prosesler



Silberschatz, Galvin and Gagne ©2018



Bölüm 3: Prosesler

- Proses Kavramı
- Proses Sıralama (Scheduling)
- Proses Üzerinde İşlemler
- Prosesler Arası İletişim
- Mesaj İletimi ve Paylaşımlı Bellek Sistemlerinde IPC
- IPC (Prosesler Arası İletişim) Örnekleri
- İstemci-Sunucu Sistemleri Arasındaki İletişim





Hedefler

- Bir prosesin ayrı bileşenlerini tanımlamak ve bunların bir işletim sisteminde nasıl temsil edildiğini ve sıralandığını görmek
- Uygun sistem çağrıları kullanarak programlar geliştirmek de dahil olmak üzere, bir işletim sisteminde proseslerin nasıl oluşturulduğunu ve sonlandırıldığını tanımlamak
- Paylaşılan bellek ve mesaj iletimini kullanarak prosesler arası iletişimi tanımlamak ve karşılaştırmak
- Prosesler arası iletişimi gerçekleştirmek için boru hatları (pipes) ve POSIX paylaşılan belleği kullanan programlar tasarlamak
- Soketler ve uzak yordam çağrıları kullanarak istemci-sunucu iletişimini tanımlamak
- Linux işletim sistemiyle etkileşime giren çekirdek modülleri tasarlamak





Proses Kavramı

- Bir işletim sistemi programları proses halinde yürütür:
- Ders kitaplarında **görev / işlem / süreç / iş (job) / proses** terimleri birbirlerinin yerine kullanılır.
- **Proses**, yürütülen bir programdır ve prosesler **sıralı bir biçimde** yürütülmelidir.
- Bir proses aşağıdakileri alanları içerir:
  - Program kodu, metin alanı (**text section**)
  - Mevcut aktiviteyi içeren **program sayacı**, proses kaydedicileri
  - Geçici veriyi tutan yığın (**stack**)
    - Fonksiyon parametreleri, döndürülen adresler, yerel değişkenler
  - Veri bölümü (**data section**) global değişkenleri tutar
  - Çalışma anında dinamik olarak tahsis edilen bellek kısmı yığıt **(heap)**





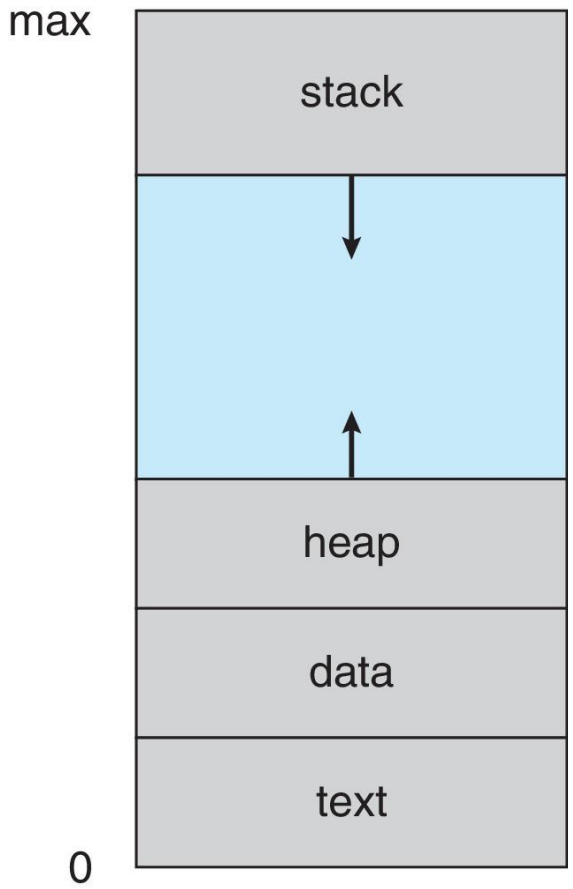
Proses

- Program çalıştırılabilir bir dosya halinde hard diskte tutulan **pasif** bir varlıktır, proses ise **aktiftir**
  - Program çalıştırılabilir halde belleğe yüklendiğinde proses halini alır.
- Programın çalıştırılması komut satırından komutun girilmesi, grafik arayüzde program ikonu üzerine tıklanması vb. ile başlar.
- Bir program birden fazla proses içerebilir.
- Örneğin birden fazla kullanıcının aynı programı çalıştırması.
  - Derleyici
  - Metin editörü





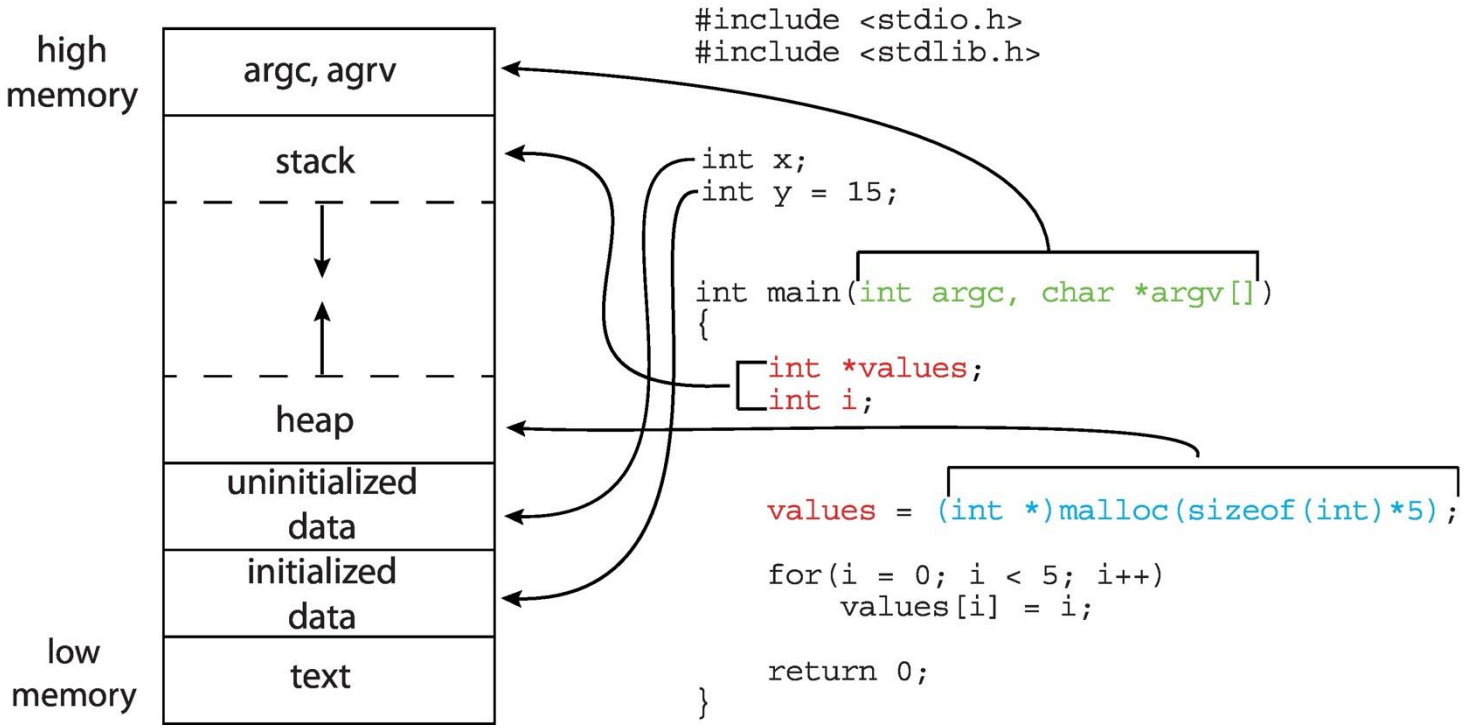
Bellekteki Bir Proses







Bir C Programının Bellek Yerleşimi







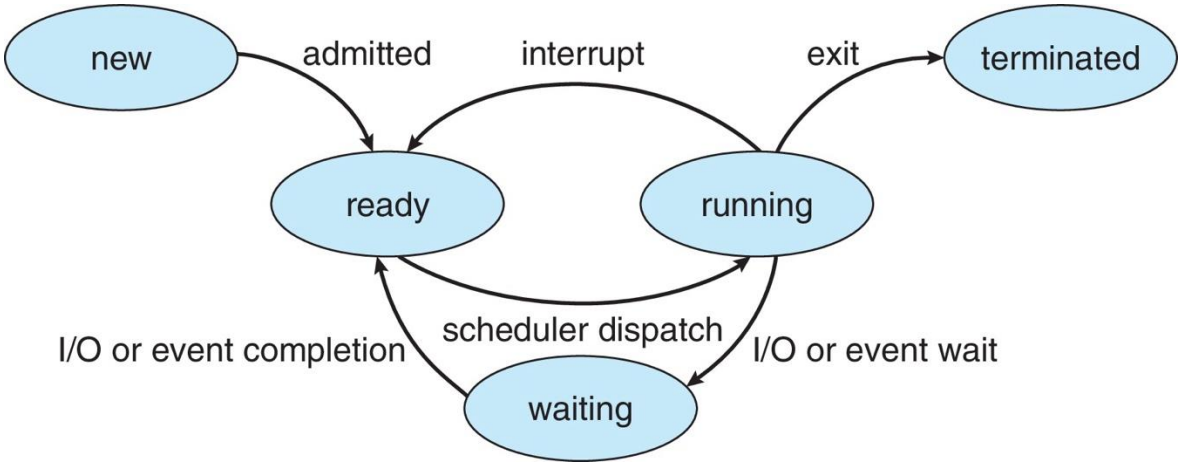
Proses Durumları

- Çalışma anında proseslerin **durumları** değişir.
  - **Yeni (New):** Yeni bir proses oluşturuluyor.
  - **Çalışıyor (Running):** İşlemler gerçekleştiriliyor.
  - **Beklemede (Waiting):** Proses bazı olayların gerçekleşmesini beklemektedir.
  - **Hazır (Ready):** Proses, işlemciye aktarılmayı beklemektedir.
  - **Sonlandırılmış (Terminated):** Prosesin yürütülmesi tamamlandığını belirtir.





Proses Durum Diyagramı



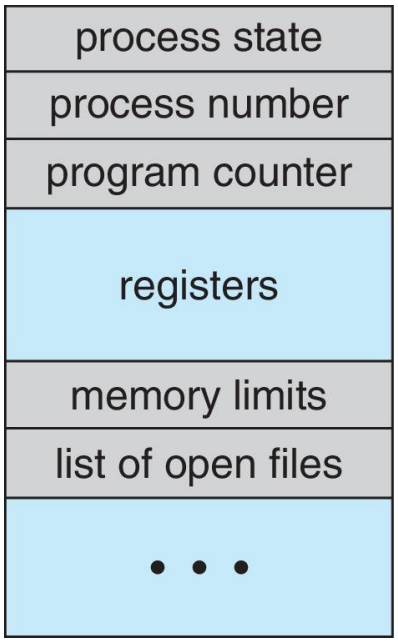




Proses Kontrol Bloğu (PCB)

Her bir proses ile ilişkili bilgi (**görev kontrol bloğu** olarakda adlandırılır) Şu bilgileri içerir :

- Proses durumu –çalışıyor, beklemede
- Program Sayacı –bir sonraki çalışacak talimatın konumu
- CPU kaydedicileri –prosese ait tüm kaydedicilerin içerikleri
- CPU sıralama bilgisi –öncelikler, sıralama kuyruk pointerları
- Bellek yönetim bilgisi –prosese tahsis edilen bellek
- Hesap bilgisi –kullanılan CPU, başlangıçtan beri geçen zaman
- I/O durum bilgisi –prosese tahsis edilen I/O aygıtları, açık dosya listesi







İş Parçacıkları (Threads)

- Şimdiye kadar, proses tek bir yürütülen iş parçacığına sahip
- Proses başına birden çok program sayacına sahip olduğumuzu düşünelim.
  - Birden fazla konum aynı anda çalışabilir
    - Birden fazla kontrol akışı -> **iş parçacıkları**
- Daha sonra iş parçacığı detayları için depolama alanı olmalı, PCB'de çoklu program sayaçları gibi
- Bölüm 4'te ayrıntılı olarak anlatılacaktır.

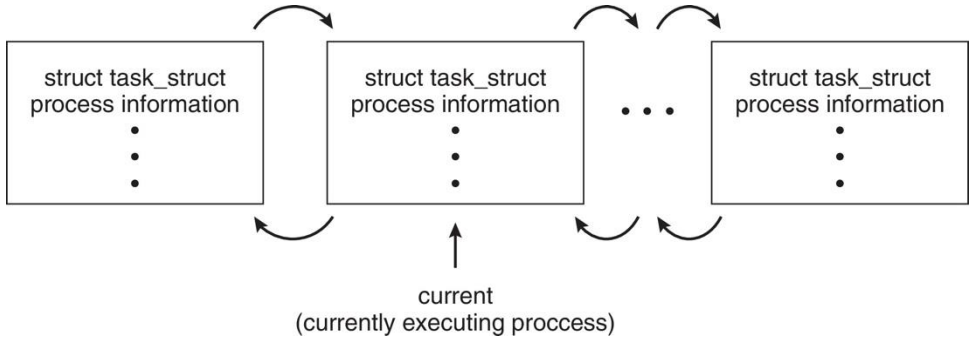




Proseslerin Linux'ta Temsili

- `task_struct` C yapısı içeriği

```
pid t_pid;                    /* Proses tanımlayıcı*/
long state;                   /* Prosesin durumu */
unsigned int time_slice       /* sıralama bilgisi */
struct task_struct *parent;/* bu prosesin ebeveyni */
struct list_head children; /* bu prosesin çocukları */
struct files_struct *files;/* açık dosyaların listesi */
struct mm_struct *mm;         /* bu prosesin adres alanı
```







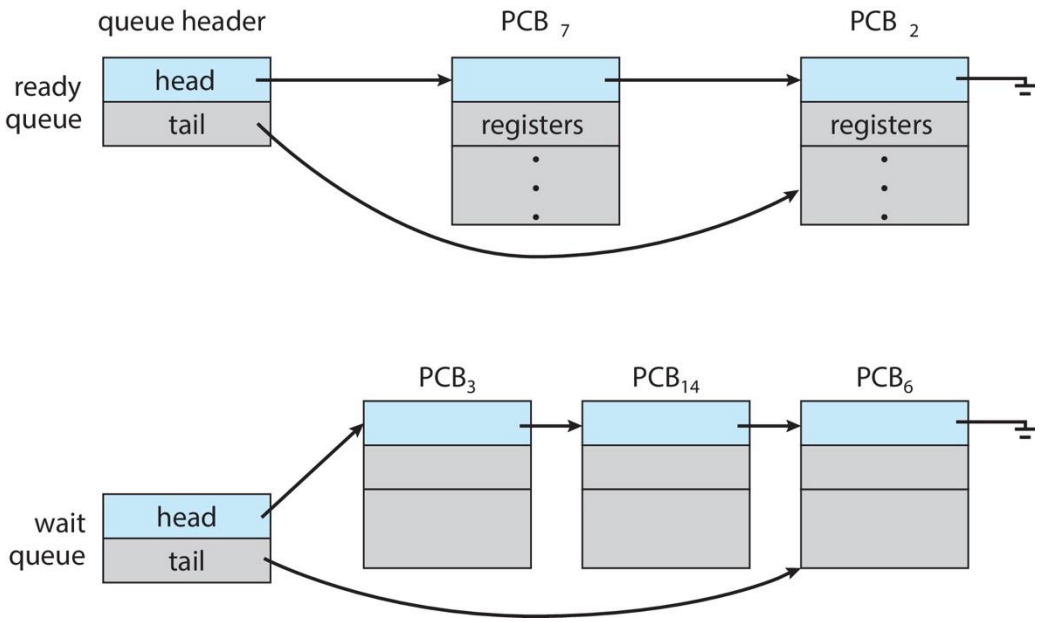
Proses Sıralama

- **Proses sıralayıcı (scheduler),** işleme hazır prosesler arasından CPU çekirdeği üzerinde işlenecek prosesi seçer.
- Hedef -İşlemciyi azami kullan, prosesler arası geçiş ve zaman paylaşımını çok hızla gerçekleştir
- Prosesler aşağıdaki **sıralama kuyruklarında** tutulur.
  - **İş kuyruğu–**sistemdeki tüm prosesler kümesi
  - **Hazır kuyruğu–**ana bellekte bulunan, hazır ve işleme girmeyi bekleyen prosesler kümesi
  - **Bekleme(Aygıt) kuyrukları–**I/O aygıtlarından gelecek mesajı bekleyen prosesler kümesi
  - Prosesler kuyruklar arasında geçiş yapabilir.





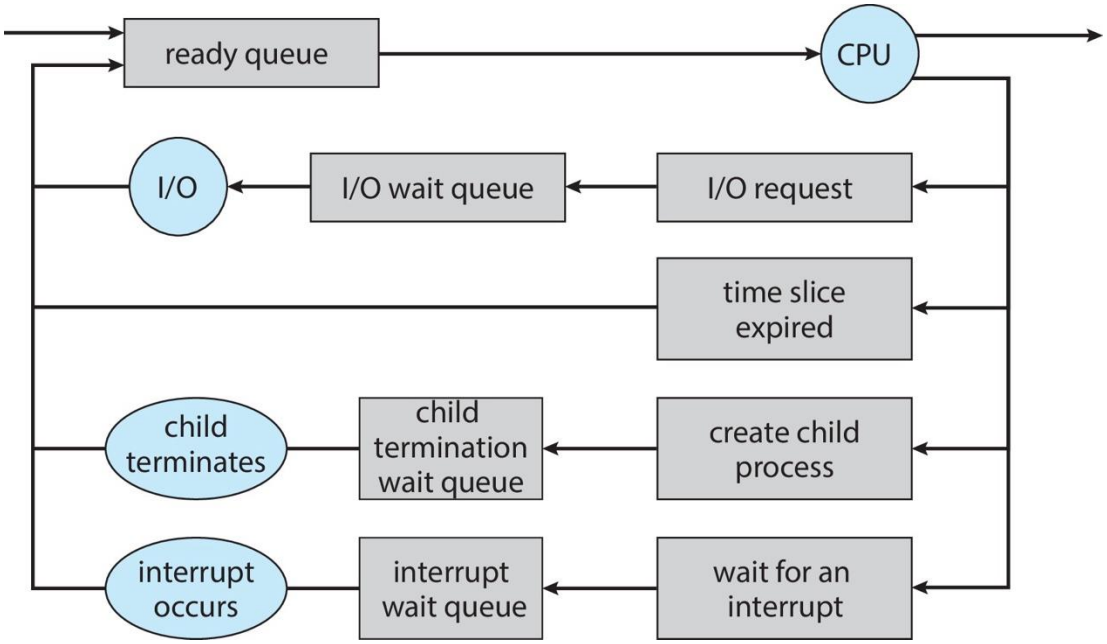
Hazır Kuyruğu ve Çeşitli I/O Aygıt Kuyrukları







Proses Sıralama Diyagramı

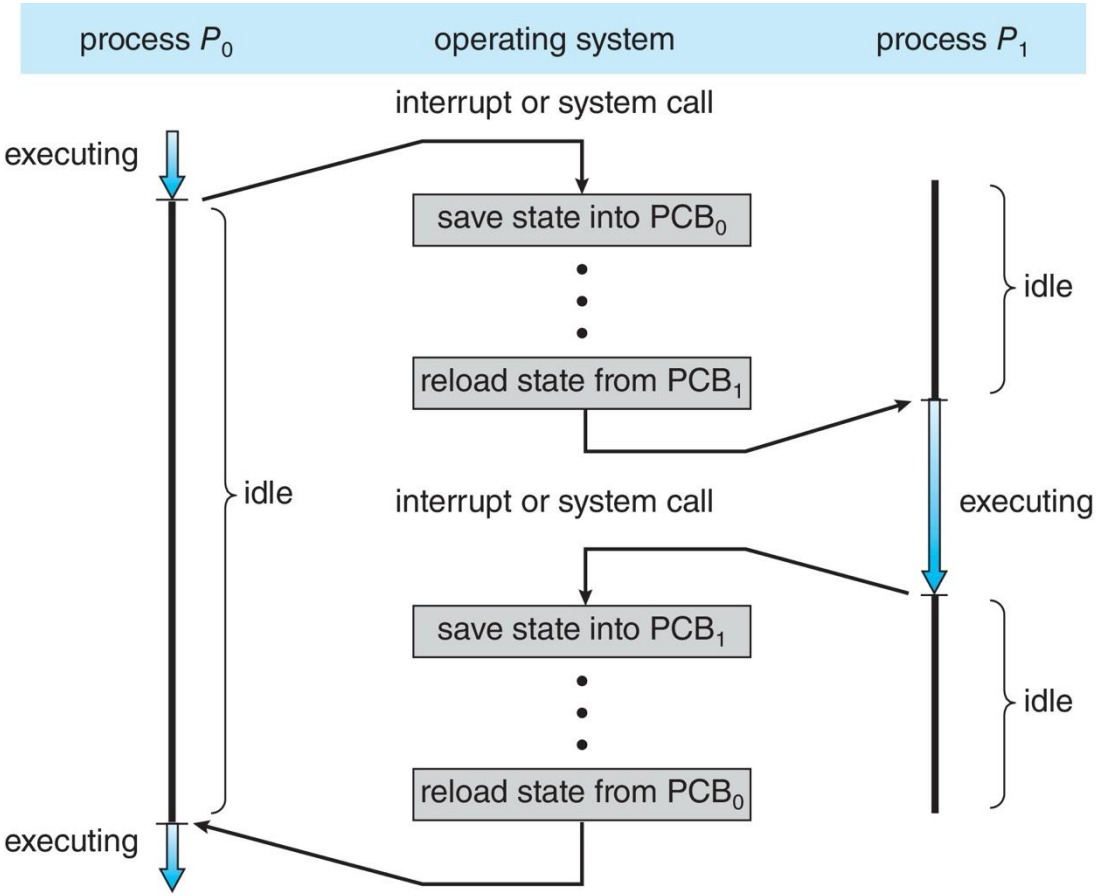






CPU'da Proses Değişimi

CPU bir prosesten diğerine geçtiğinde bir bağlam değişimi **(contextswitch)** meydana gelir







Bağlam Değişimi (Context Switch)

- CPU diğer prosese geçtiği zaman, sistem mutlaka eski prosesin **durumunu kaydetmeli** ve **bağlam değişimi** yoluyla yeni prosesin daha önce **kaydedilmiş durumunu** yüklemeli
- Bir prosesini **bağlamı (context)** PCB ile temsil edilir
- Bağlam değiştirme bir maliyettir; sistem geçişler sırasında kullanım dışıdır
  - Daha karmaşık OS ve PCB -> daha uzun bağlam değişimi
- Donanım desteği zamana bağlıdır.
  - Bazı donanımlar CPU başına birden fazla kaydedici sağlar -> birden fazla bağlam bir kerede yüklenir.





Proses İşlemleri

- Sistem aşağıdaki işlemler için mekanizmalar sağlamalıdır :
  - Proseslerin oluşturulması
  - Proseslerin sonlandırılması





Proses Oluşturulması

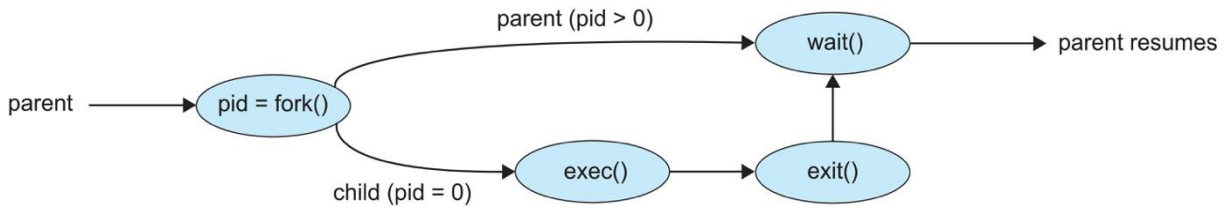
- **Ebeveyn** Proses, **çocuk** prosesleri oluşturur. Bu şekilde ağaç yapısı meydana gelir.
- Genelde prosesler, **bir proses kimlik numarası** (Proses identifier -**pid**) ile tanımlanır ve yönetilir.
- Kaynak Paylaşımı türleri
  - Ebeveyn ve çocuk prosesler tüm kaynakları paylaşır.
  - Çocuk prosesler ebeveyn prosesin kaynaklarını kullanır.
  - Ebeveyn ve çocuk hiçbir kaynağı paylaşmaz.
- Uygulama:
  - Ebeveyn ve çocuk proses eş zamanlı çalışır
  - Ebeveyn proses, çocuk proses sonlanana kadar bekler





Proses Oluşturulması (Devam)

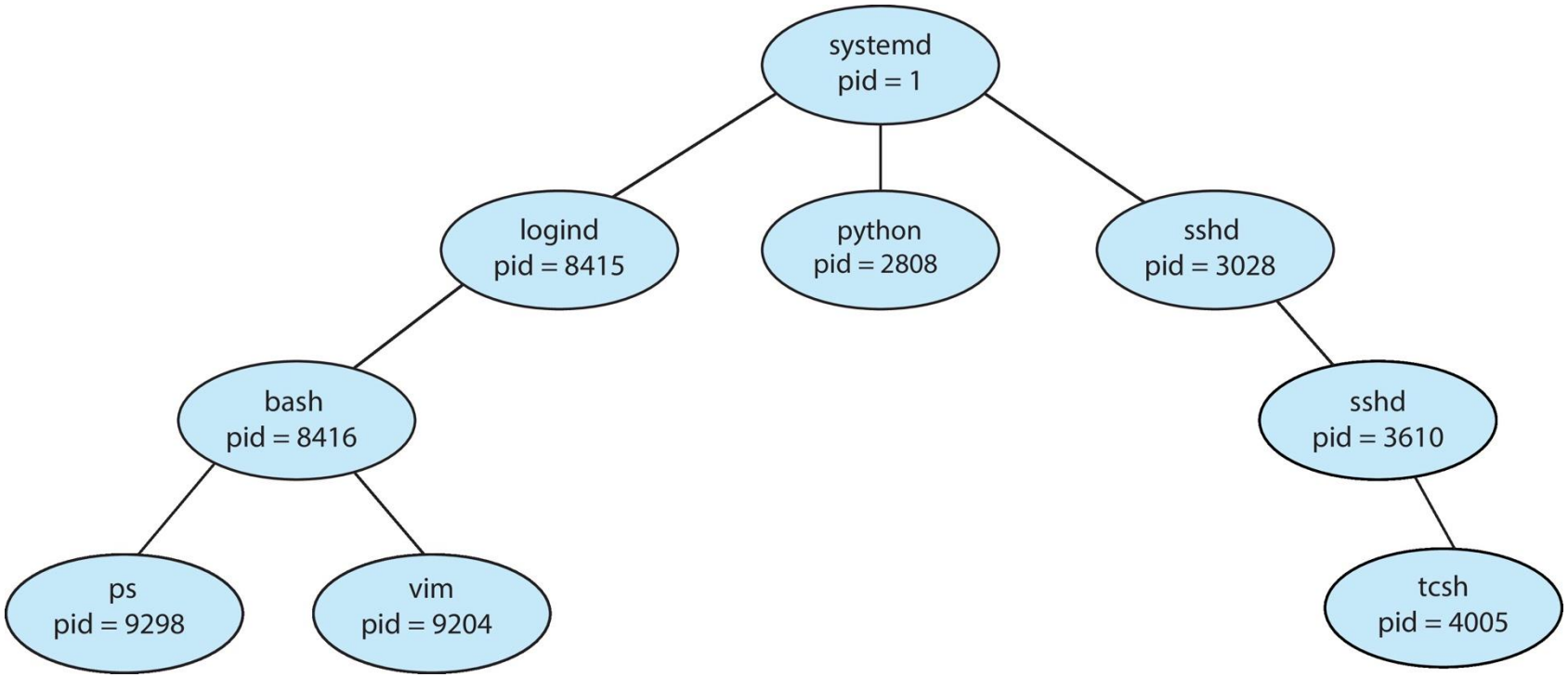
- Adres alanı
  - Çocuk proses, ebeveyn prosesin alanını kopyalar .
  - Çocuk prosese bir program yüklenmiş olur.
- UNIX örnekleri
  - **fork()** sistem çağrısı yeni bir proses oluşturur
  - **exec()** sistem çağrısı prosesin bellek alanını yeni bir program ile değiştirmek için bir **fork()** sonrası çağrılır.
  - Ebeveyn proses çocuğun sonlanmasını beklemek için **wait()** çağırır







Linux Proses Ağacı







Fork İşlemi Yapan C Programı

```
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>

int main()
{
pid_t pid;

    /* fork a child process */
    pid = fork();

    if (pid < 0) { /* error occurred */
       fprintf(stderr, "Fork Failed");
       return 1;
    }
    else if (pid == 0) { /* child process */
       execlp("/bin/ls","ls",NULL);
    }
    else { /* parent process */
       /* parent will wait for the child to complete */
       wait(NULL);
       printf("Child Complete");
    }

    return 0;
}
```





Proses Sonlanması

- Proses son kod ifadesinin çalıştırır ve işletim sistemine **exit ()** sistem çağrısını kullanarak silinmesini talep eder
  - Durum verisi çocuktan ebeveyn prosese döner (**wait () aracılığıyla**)
  - Prosese ayırılan alan işletim sistemi tarafından geri alınır
- Ebeveyn **abort** () sistem çağrısını kullanarak çocuk proseslerin çalışmasını sonlandırabilir. Bunun sebepleri:
  - Çocuk tahsis edilen kaynakların dışına çıkmıştır.
  - Çocuk prosese tayin edilen iş artık gerekli değildir
  - Ebeveyn prosesten çıkılır, ve işletim sistemi ebeveyn proses sonlandırıldıktan sonra çocuk prosesin çalışmasına izin vermez



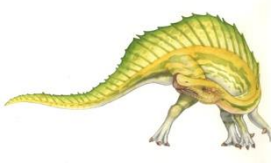


Proses Sonlanması

- Bazı işletim sistemleri, ebeveyni sonlandırıldıysa çocuk prosesin var olmasına izin vermez. Bir proses sona ererse, tüm çocuklarıda sonlandırılmalıdır.
  - **Basamaklı sonlandırma**. Tüm çocukların, torunların vs. çalışmasına son verilir.
  - Sonlandırma işletim sistemi tarafından başlatılır.
  - Ebeveyn proses **wait()** sistem çağrısını kullanarak bir çocuk prosesin sonlandırılmasını bekleyebilir. Çağrı durum bilgilerini ve sonlandırılan prosesin pid'sini döndürür

pid= wait(&status);

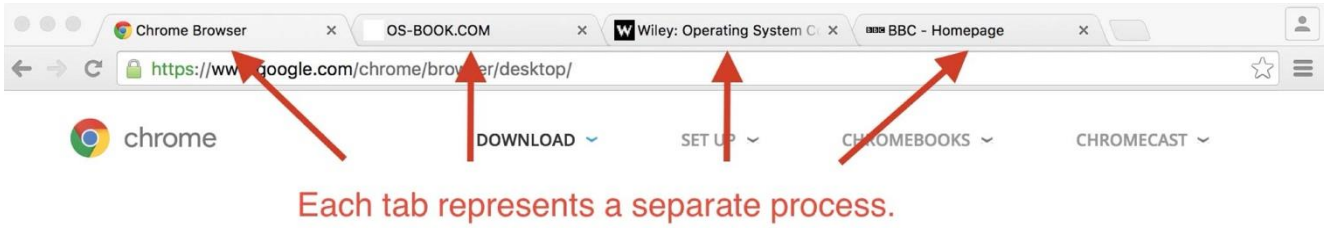
- Bekleyen hiçbir ebeveyn yoksa (wait() çağrılmadıysa) proses bir **zombidir.**
- Ebeveyn wait() çağrılmadan sonlandırıldıysa, proses bir **artıkdır (orphan)**





Çoklu Proses Mimarisi –Chrome Tarayıcısı

- Bir çok web tarayıcısı tek proses olarak çalıştırılır (bazıları hala öyle)
  - Bir web sitesi soruna neden olursa, tüm tarayıcı askıda kalabilir veya çökebilir
- Google Chrome Tarayıcı 3 farklı proses türüyle çoklu bir proses yapısındadır:
  - **Tarayıcı** prosesi kullanıcı arayüzünü yönetir, disk ve ağ G/Ç
  - **İşleyici (Renderer)** prosesi web sayfalarını işler, HTML, Javascript ile ilgilenir. Her web sitesi için oluşturulan yeni bir işleyici bulunur
  - Disk ve ağ G/Ç'yi kısıtlayan, güvenlik açıklarının etkisini en aza indiren korumalı (**sandbox**) bir yapıda çalışır
- Her eklenti türü için **Plug-in** prosesi







Prosesler Arası İletişim

- Prosesler işletim sistemi içerisinde **bağımsız** ya da **işbirliği halinde** çalışabilirler.
- İşbirliği içerisindeki prosesler veri paylaşımı da dahil olmak üzere diğer prosesleri etkileyebilir ya da diğer proseslerden etkilenebilirler.
- Proseslerin işbirliği yapma nedenleri:
  - Bilgi paylaşımı
  - Daha hızlı hesaplama
  - Modülerlik
  - Rahatlık
- İşbirliği içindeki prosesler **prosesler arası haberleşmeye (Interproses communication - IPC)** ihtiyaç duyarlar.
- 2 temel IPC modeli mevcuttur:
  - **Paylaşımlı bellek** (kullanıcıların kontrolünde)
  - **Mesaj Gönderimi**(OS nin kontrolünde)

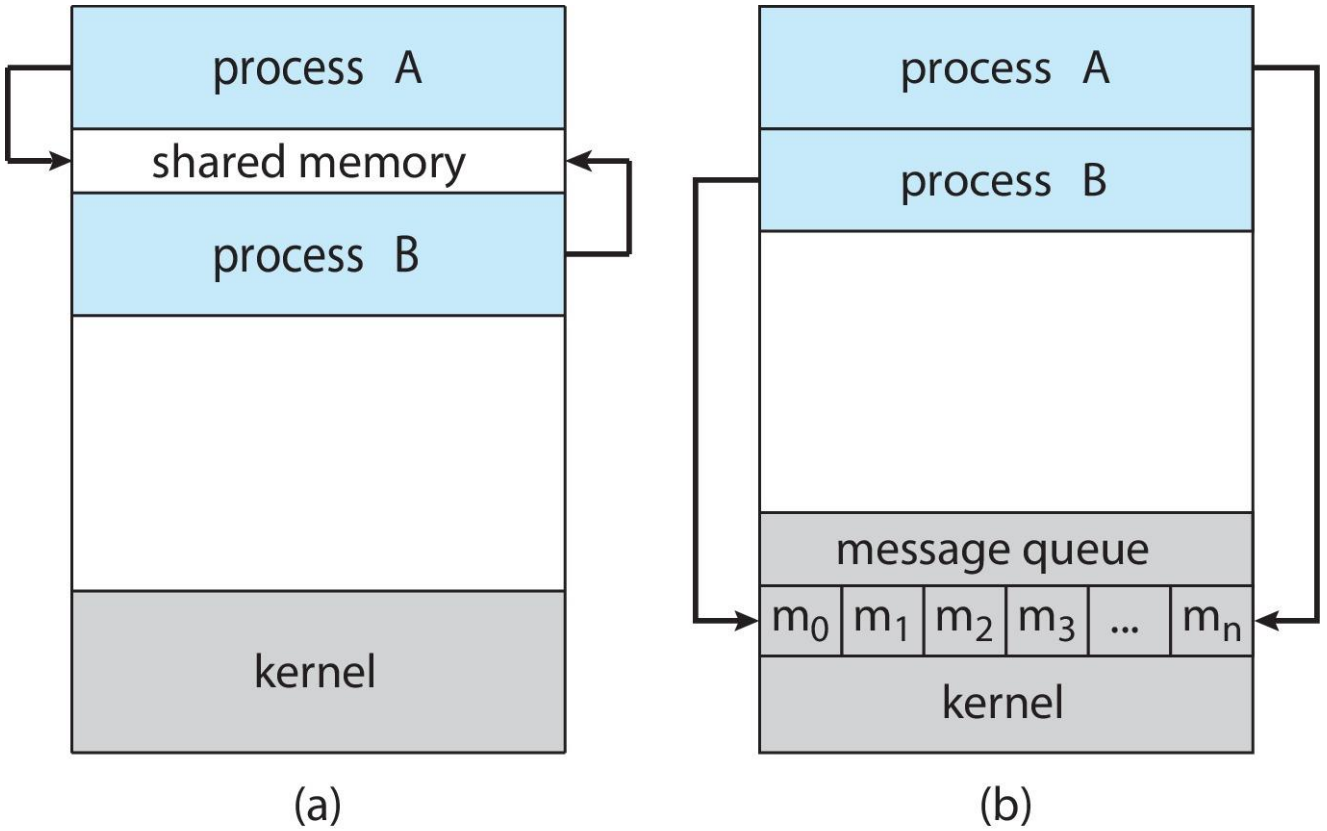




Haberleşme Modelleri

(a) Shared memory.

(b) Message passing.







Üretici-Tüketici Problemi

- İşbirliği içindeki proseslere ilişkin bir paradigma:
  - **Producer (üretici)** veri üretir, **Consumer (tüketici)** bu veriyi tüketir.
  - Bu iki süreç aynı **shared buffer** üzerinden iletişim kurar.
- İki versiyonu vardır:
  - **Sınırlandırılmamış tampon**: Tampon için limit konulmamıştır
    - Üretici asla beklemez
    - Tüketici tamponda ürün yoksa bekler
  - **Sınırlı tampon**: Sabit bir tampon boyutu mevcuttur.
    - Üretici eğer tampon dolu ise beklemek zorunda
    - Tüketici tamponda ürün yoksa bekler
- Producer ve Consumer aynı anda buffer'a erişirse çakışma (race condition) olabilir.





Paylaşımlı Bellek Çözümü

- İki veya daha fazla sürecin, **ortak bir bellek bölgesine erişerek veri paylaşmasıdır.**
  Bu bellek alanı, süreçler tarafından birlikte kullanılır ve bu bölgeye erişim işletim sistemi tarafından kısıtlanmaz.

Özellikler:

- Başlangıçta, bellek alanı bir süreç tarafından oluşturulur.
- Diğer süreçler bu bölgeyi kendi **address space'**lerine "attach" eder.
- Veri yapısı ve erişim düzeni programcı tarafından belirlenir (OS tarafından değil).
- Süreçler, aynı anda aynı alana yazmamak için **senkronizasyon (synchronization)** sağlamalıdır.





Sınırlı-Tamponlu Paylaşımlı-Bellek Çözümü

- Paylaşılan veriler

```
#define BUFFER_SIZE 10
    typedef struct {
        . . .
    } item;

    item buffer[BUFFER_SIZE];
    int in = 0;
    int out = 0;
```

- **Paylaşımlı bellek (shared memory)** üzerinde **üretici (producer)** ve **tüketici (consumer)** süreçlerin **veri alışverişini** düzenli bir şekilde yapmasını sağlar.
- Üretici veri üretip buffer'a yazar, tüketici buffer'daki veriyi okur (tüketir).
- Bu işlem sırasında **yarış koşulu (race condition)** oluşmaması gerekir.





Sınırlı-Tampon – Üretici

```
item next_produced;

while (true) {
   /* next_produced içinde bir öğe üret */
   while (((in + 1) % BUFFER_SIZE) == out)
        ; /* do nothing: buffer doluysa bekle */
   buffer[in] = next_produced; // öğeyi yaz
   in = (in + 1) % BUFFER_SIZE; // dairesel ilerle
}
```





Sınırlı Tampon – Tüketici

```
item next_consumed;

while (true) {
        while (in == out)
                ; /* do nothing: buffer boşsa bekle */
        next_consumed = buffer[out]; //sıradaki öğeyi
al
        out = (out + 1) % BUFFER_SIZE; //dairesel
ilerle

        /* consume the item in next consumed : öğeyi
tüket */
}
```





Tamponlar dolduğunda ne olur?

- **Tüm** tamponları dolduran tüketici-üretici sorununa bir çözüm sunmak istediğimizi varsayalım.
- Bunu, tam tampon sayısını izleyen bir tamsayı sayacı ile yapabiliriz.
- Başlangıçta **sayaç 0** olarak ayarlanır.
- Tamsayı **sayacı**, yeni bir tampon elemanı ürettikten sonra üretici tarafından artırılır.
- Tamsayı **sayacı**, tampon elemanını tükettikten sonra tüketici tarafından azaltılır.





Üretici

```
while (true) {
        /* produce an item in next produced */


        while (counter == BUFFER_SIZE)  // Tampon
dolu
                ; /* do nothing */
        buffer[in] = next_produced;
        in = (in + 1) % BUFFER_SIZE;
        counter++;
}
```





Tüketici

```
while (true) {
        while (counter == 0)
                ; /* do nothing */ //Tampon bomboş
        next_consumed = buffer[out];
        out = (out + 1) % BUFFER_SIZE;
         counter--;
        /* consume the item in next consumed */
}
```





Yarış Durumu

- **counter++** aşağıdaki gibi çalışır

  register1 = counter
  register1 = register1 + 1
  counter = register1

- **counter--** aşağıdaki gibi çalışır

  register2 = counter
  register2 = register2 - 1
  counter = register2

- Başlangıçta "count = 5" ile bu yürütme sırasını ele alalım:

  S0: producer execute **register1 = counter** {register1 = 5}
  S1: producer execute **register1 = register1 + 1** {register1 = 6}
  S2: consumer execute **register2 = counter** {register2 = 5}
  S3: consumer execute **register2 = register2 – 1** {register2 = 4}
  S4: producer execute **counter = register1** {counter = 6 }
  S5: consumer execute **counter = register2** {counter = 4}





IPC - Prosesler Arası İletişim Mesajlaşma

- Proseslerin iletişim mekanizması diğer proseslerin işlemleriyle senkronizedir.
- Prosesler aynı bellek alanını paylaşmadan, **mesaj gönderme (send)** ve **alma (receive)** fonksiyonlarıyla iletişim kurar.
- IPC iki işlemi destekler :
  - send(message)
  - receive(message)
- Gönderilecek mesaj boyutu, sabit ya da değişken olabilir.

Avantajlar:

- Farklı makinelerdeki süreçler arasında da çalışabilir.
- Dağıtık sistemlerde (distributed systems) uygundur.
- Küçük veri transferlerinde verimlidir.

Dezavantaj:

- Her mesaj **kernel üzerinden geçtiği için** system call overhead'i vardır.





Mesajlaşma (devam)

- Eğer P ve Q prosesleri iletişim kurmak istiyorsa, şu işlemleri yapmaları gerekir :
  - Aralarında **iletişim bağlantısı** var olmalıdır.
  - send /receive yardımı ile mesaj alışverişi gerçekleştirmelidirler.
- Uygulama sorunları:
  - Bağlantılar nasıl kurulur?
  - Bir bağlantı ikiden fazla prosesle ilişkilendirilebilir mi?
  - Her iletişim proses çifti arasında kaç bağlantı olabilir?
  - Bağlantının kapasitesi nedir?
  - Bağlantının sabit veya değişken bir iletinin boyutu mu?
  - Bağlantı tek yönlü mü yoksa çift yönlü mü?





Bir İletişim Bağlantısının Uygulanması

- Fiziksel
  - Paylaşımlı bellek
  - Donanım veri yolu
  - Ağ
- Mantıksal
  - Doğrudan veya dolaylı
  - Senkron veya asenkron
  - Otomatik veya açık tamponlu





Doğrudan İletişim

- Proseslerin her biri gönderici ve alıcı olarak isimlendirilmelidir :
  - **send** (P, message) – P prosesine mesaj gönder
  - **receive**(Q, message) – Q prosesinden mesaj al
- İletişim bağlantısının özellikleri
  - Bağlantılar otomatik olarak kurulur
  - Her bir proses çifti arasında tam olarak bir bağlantı vardır.
  - Bir link 2 proses ile ilişkilendirilebilir.
  - Bağlantı tek yönlü olabilir, ancak genellikle çift yönlüdür





Dolaylı İletişim

- Mesajlar port veya posta kutularından alınır veya buralara gönderilir.
  - Her posta kutusu tek bir tanımlayıcıya sahiptir.
  - Prosesler paylaşılmış bir posta kutusuna sahipse iletişim kurabilirler.

- İletişim bağlantısı özellikleri şunlardır :
  - Bağlantı, prosesler arası paylaşılmış bir posta kutusu var ise kurulur.
  - Bir bağlantı ikiden fazla proses ile ilişkilendirilebilir.
  - Her bir proses çifti birden fazla bağlantıya sahip olabilir.
  - Bağlantı tek yönlü ya da çift yönlü olabilir.





Dolaylı İletişim

- İşlemler
  - Yeni bir posta kutusu oluştur,
  - Posta kutusu aracılığıyla mesaj gönder ve al.
  - Posta kutusunu yok et.

- İletişim basitçe şu şekilde gerçekleşir:

  send(A, message) – A'nın posta kutusuna bir mesaj gönder

  receive(A, message) – A'nın posta kutusundan bir mesaj al.





Dolaylı İletişim

- Dolaylı İletişim
  - *P1 , P 2 ve P 3 prosesleri A posta kutusunu paylaşıyor.*
  - *P 1 mesaj gönderiyor; P 2 ve P 3 mesajı alıyor.*
  - *Mesajı hangisi almıştır?*
- Çözüm:
  - Bir bağlantının en fazla iki proses ile ilişkilendirilmesine izin ver.
  - Bir seferde yalnızca bir proses yürütmesine izin ver.
  - Sistemin rastgele bir alıcı seçimine izin ver. Gönderici, alıcının kim olduğunu bildirir.





Senkronizasyon

Mesaj iletimi engelli ya da engelsiz olabilir.

- **Engelli, senkron** iletim olarak düşünülebilir.
  - **Engelli gönderim**, mesaj alınana kadar gönderici engellenir.
  - **Engelli alım**, mesaj hazır olana kadar alıcı engellenir.
- **Engelsiz, asenkron** iletim olarak düşünülebilir.
  - **Engelsiz gönderim**, mesaj yollanır ve devam edilir.
  - **Engelsiz alım**, hazır mesaj varsa alır yoksa boş-null değer alır.
- Farklı kombinasyonlar mümkün
  - Hem gönderme hem de alma engelliyorsa, canlı yayın (rendezvous)





Üretici-Tüketici: Mesajlaşma

- Üretici

```
message next_produced;
while (true) {
  /* produce an item in next_produced */

  send(next_produced);
}
```

- Tüketici

```
message next_consumed;
while (true) {
   receive(next_consumed)

  /* consume the item in next_consumed */
}
```





Tamponlama

- Bağlantıyla ilişkilendirilmiş mesaj kuyruğu,
- 3 şekilde uygulanır:
  - 1. Sıfır kapasite – 0 mesaj

    Gönderici, alıcıyı beklemelidir (Buluşma).
  - 2. Sınırlı kapasite –n adet mesajın sonlu bir uzunluğa sahip olması

    Gönderici, bağlantı dolu ise beklemelidir.
  - 3. Sınırsız kapasite – sonsuz uzunluk

    Gönderici hiçbir zaman beklemez.





Veri Kanalları- Pipes

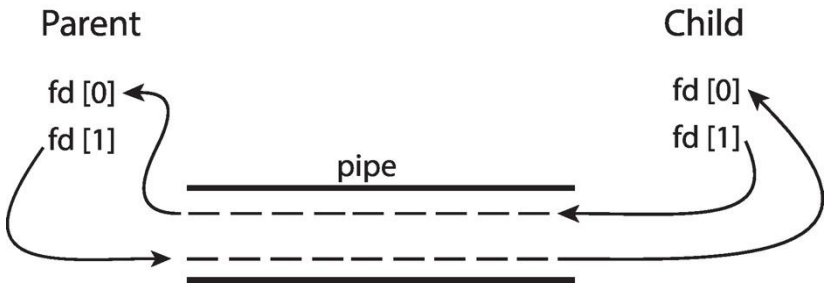
- İki proses arasında iletişime izin veren yapıdır.
- Sorunlar:
  - İletişim tek yönü mü, çift yönlü müdür?
  - İletişim iki yönlü ise yarı dubleks mi çalışır, yoksa tam dubleks mi çalışır?
  - İletişim halindeki prosesler arasında bir ilişki (ebeveyn-çocuk) olmalı mıdır?
  - Kanallar ağ üzerinden kullanılabilir mi?
- **Sıradan kanallar** –oluşturan prosesin dışından erişilemez. Genellikle, bir ebeveyn proses bir kanal oluşturur ve çocuk prosesi ile iletişim kurmak için kullanır.
- **İsimli kanallar** – üst-alt ilişkisi olmadan erişilebilir.





Sıradan Kanallar

- **Sıradan kanallar**, standart üretici-tüketici tipi iletişime izin verir.
- Üretici bir uçtan yazar (kanalın yazma ucu)
- Tüketici diğer ucundan okur (kanalın okuma ucu)
- Sıradan kanallar bu nedenle tek yönlü iletişim sağlar.
- Haberleşen prosesler arasında ebeveyn-çocuk ilişkisi gerekir.



- Windows bunları **anonim kanallar** olarak adlandırır





İsimli Kanallar

- İsimli Kanallar, sıradan olanlardan daha güçlüdür.
- İletişim çift yönlüdür.
- Haberleşen prosesler arasında ebeveyn-çocuk ilişkisi gerekli değildir.
- Birden fazla proses kullanabilir.
- UNIX ve Windows işletim sistemlerince desteklenir.





İstemci – Sunucu Sistemlerinde İletişim

- Soketler
- Uzaktan Prosedür Çağrıları
- Uzaktan Metot Çağrıları (RMI - Java)





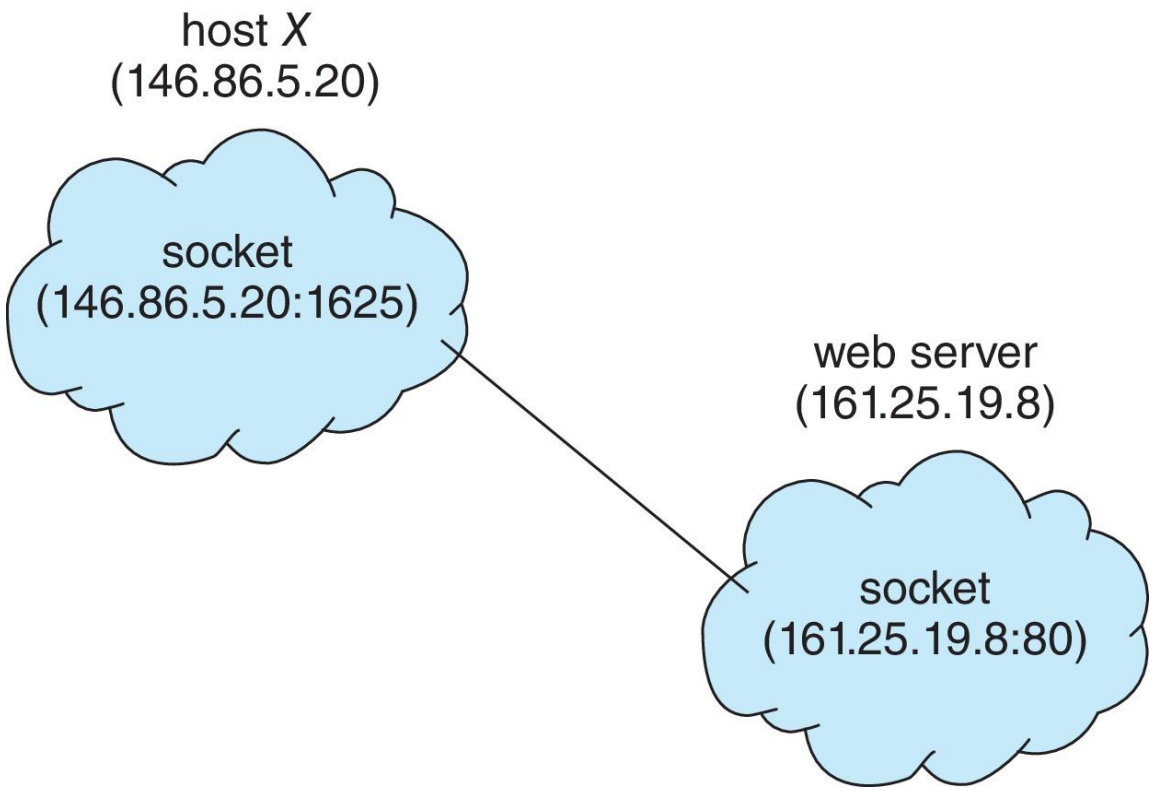
Soketler

- Bir **soket**, bir iletişim uç noktası olarak tanımlanabilir.
- IP adresinin ve **portun** birleşimidir.
  - Port, bir ana bilgisayardaki ağ hizmetlerini ayırt etmek için mesaj paketinin başında bulunan bir sayıdır
- **161.25.19.8:1625** soketi, **1625** portu ve **161.25.19.8** sunucusu demektir.
- İletişim, bir çift soket arasında meydana gelir.
- 1024'ün altındaki tüm port numaraları iyi bilinmektedir, standart hizmetler için kullanılır
- Prosesin çalıştığı sisteme başvurmak için özel IP adresi 127.0.0.1 (geridöngü-**loopback**)



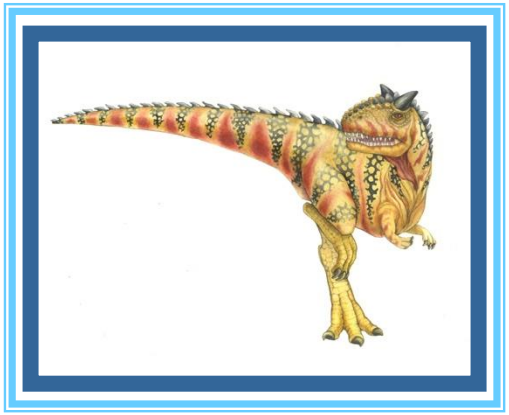


Soket İletişimi





Bölüm 3 - Son



Silberschatz, Galvin and Gagne ©2018